Indexing in File Organization

Lecture 3: Ordered, Dense, Sparse, Multilevel, Inverted and Hashed Indices

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: File Organization

In previous lectures, we studied:

- Sequential file organization.
- Relative file organization.
- Indexed sequential file organization.
- Organization of records into blocks.
- Blocking factor and block utilization.

Today's Problem

How can we locate a particular record quickly without examining every record in a large file?

Why Do We Need Indexing?

Consider a file containing:

$$n = 1,000,000 \text{ records}$$

Without an index, searching an unsorted file may require examining all records.

$$T_{\text{search}} = O(n)$$

With an Index

An auxiliary data structure helps identify the location of the required record.
This can significantly reduce retrieval cost.

What Is an Index?

Definition

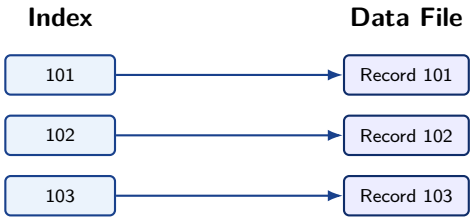
An index is an auxiliary structure containing search-key information and references to corresponding records or blocks.

- An index helps locate records efficiently.
- A file may have multiple indices.
- Index entries may store keys and pointers.
- Indices require additional storage.

Analogy

A library catalogue helps locate books without searching every shelf.

Basic Structure of an Index



Observation

The index maps a search key to a record location.

Index Entry

A typical index entry contains:

$$\boxed{(\text{Search Key}, \text{Pointer})}$$

Search Key	Record Address
101	1200
102	1450
103	1780
104	2020

Example

For key 103, the index returns address 1780.

Benefits of Indexing

1. Faster record retrieval.
2. Reduced full-file scanning.
3. Support for searching on different fields.
4. Improved interactive query performance.
5. Efficient access to large files.

Trade-off

Indices consume additional space and must be maintained when records change.

Factors for Choosing an Index

According to the shared notes, indexing techniques can be evaluated using:

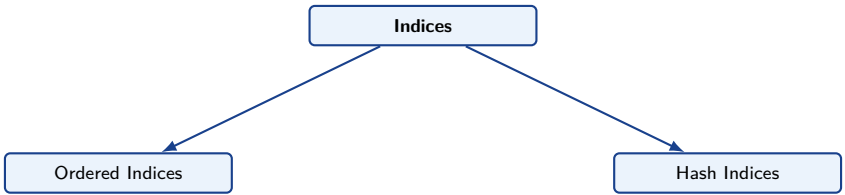
- Access type.
- Access time.
- Insertion time.
- Deletion time.
- Space overhead.

Design Principle

Choose the index based on the operations most frequently performed by the application.

Major Types of Indices

The notes classify indices into two major groups.



- Ordered indices maintain sorted search keys.
- Hash indices use a hash function to determine bucket locations.

Ordered Indices

Definition

An ordered index maintains its search-key entries in sorted order.

Key	Pointer
10	P1
20	P2
30	P3
40	P4
50	P5

- Supports ordered traversal.
- Can support range queries.
- May be searched using suitable algorithms.

Hashed Indices

Definition

A hashed index uses a hash function to determine the bucket containing the desired record.

$$h(k) \longrightarrow \text{Bucket Address}$$

Example

$$h(k) = k \bmod 5$$

$$h(23) = 23 \bmod 5 = \boxed{3}$$

The record with key 23 is assigned to bucket 3.

Primary Index

Definition

In a sequentially ordered file, the index whose search key determines the file's sequential order is called the primary index.

Example

Student records are ordered by roll number.
An index on roll number is the primary index.

- Corresponds to the ordering field.
- Supports searching by the ordering key.
- Common in ordered file organizations.

Primary Index: Example

Roll Number	Name	Marks
101	Aman	85
102	Riya	91
103	Neha	87
104	Rahul	79
105	Simran	94

Observation

The data file is ordered by roll number.
Therefore, an index based on roll number is a primary index.

Secondary Index

Definition

A secondary index uses a search key different from the field determining the sequential order of the file.

Example

A student file is ordered by roll number.
An index on student name is a secondary index.

- Supports queries on non-ordering fields.
- A file may have several secondary indices.
- Secondary search keys may be non-unique.

Secondary Index: Example

Roll Number	Name	Marks
101	Aman	85
102	Riya	91
103	Neha	87
104	Rahul	79
105	Simran	94

Secondary Index on Name

Aman → Record 101
Neha → Record 103
Rahul → Record 104
Riya → Record 102
Simran → Record 105

Primary vs Secondary Index

Feature	Primary Index	Secondary Index
Search key	Ordering field	Non-ordering field
File order	Matches search-key order	Different from search-key order
Typical example	Roll number	Student name
Number	One physical ordering field	Multiple secondary indices possible
Purpose	Search by ordering key	Search by other fields

Dense Index

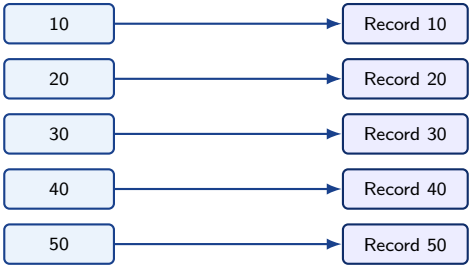
Definition

A dense index contains an index entry for every search-key value or record in the indexed data, as appropriate to the index design.

In the textbook's example, an entry is maintained for every record.

- Provides direct index coverage.
- Requires more index entries.
- May need more maintenance.

Dense Index: Diagram



Observation

Every displayed record has a corresponding index entry.

Sparse Index

Definition

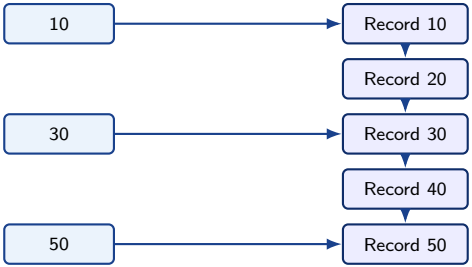
A sparse index contains entries for only selected search-key values or record locations.

- Requires less index storage.
- Depends on ordered data for efficient lookup.
- Searching may require scanning after locating a nearby indexed position.

Example

Only keys 10, 30, and 50 are indexed, even though the file also contains 20 and 40.

Sparse Index: Diagram



Observation

The index stores pointers to selected records.

Searching Using a Sparse Index

Problem

Data keys:

$$10,\ 20,\ 30,\ 40,\ 50,\ 60$$

Sparse index keys:

$$10,\ 30,\ 60$$

Search for key 40.

1. Find the largest indexed key ≤ 40.
2. The matching index entry is 30.
3. Follow its pointer to record 30.
4. Scan forward to record 40.

$$\boxed{30 \rightarrow 40}$$

Sparse Index: Search Algorithm

1. Read the target search key K.
2. Search the sparse index.
3. Find the largest indexed key not exceeding K.
4. Follow the corresponding pointer.
5. Search forward through ordered records.
6. Stop when the key is found or when the current key exceeds K.

Requirement

The file must be appropriately ordered for this search method to work.

Dense vs Sparse Index

Feature	Dense Index	Sparse Index
Entries	Every record or key	Selected records or keys
Index size	Larger	Smaller
Lookup	Direct index coverage	May require local scan
Maintenance	Potentially more updates	Potentially fewer updates
Ordered file	Not always required	Normally required

Numerical Example: Index Size

Given

- Number of records = 10,000
- Index entry size = 12 bytes
- Dense index: one entry per record

$$\text{Index Size} = 10000 \times 12$$

$$= \boxed{120000 \text{ bytes}}$$

Sparse Index

If one entry represents each of 100 data blocks:

$$100 \times 12 = \boxed{1200 \text{ bytes}}$$

Cylinder-Surface Indexing

Definition

Cylinder-surface indexing is a technique for indexing sequentially ordered records based on traditional magnetic disk geometry.

- Used with the primary ordering key.
- Organizes indexing at cylinder and surface levels.
- Helps identify the track containing a record.

Context

This is a historical disk-oriented technique described in the shared textbook.

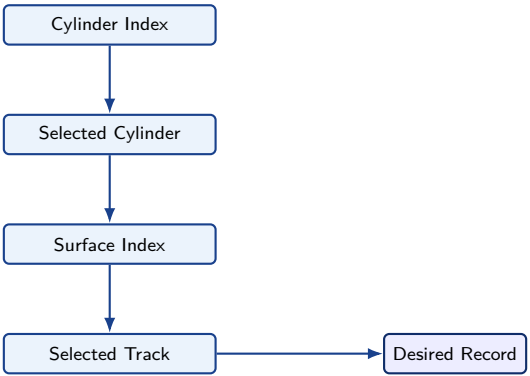
Cylinder and Surface

- A magnetic disk may have multiple recording surfaces.
- Each surface contains concentric tracks.
- Tracks at the same radius form a cylinder.
- A cylinder index identifies the relevant cylinder.
- A surface index identifies the relevant track.

Indexing Hierarchy

$$\text{Cylinder Index} \rightarrow \text{Surface Index} \rightarrow \text{Track} \rightarrow \text{Record}$$

Cylinder-Surface Indexing: Diagram



Cylinder-Surface Search Steps

1. Read the cylinder index.
2. Search for the appropriate cylinder.
3. Retrieve its surface index.
4. Determine the relevant track.
5. Read the track.
6. Locate the desired record.

Textbook I/O Model

The shared notes describe three principal disk accesses: cylinder index, surface index, and track retrieval.

Cylinder-Surface Index: Numerical

Suppose a file occupies:

$$m = 20 \text{ cylinders}$$

Each cylinder has:

$$n = 6 \text{ usable surfaces}$$

Cylinder-index entries:

$$\boxed{20}$$

Total surface-index entries:

$$m \times n = 20 \times 6$$

$$\boxed{120}$$

Indexed Sequential Access Method

ISAM

The shared textbook associates cylinder-surface indexing with the Indexed Sequential Access Method.

- Combines ordered storage and indexing.
- Supports efficient retrieval by the ordering key.
- May use multiple levels of index information.
- Primarily associated with disk-based file systems.

Limitation

A primary physical ordering cannot simultaneously match several unrelated search keys.

Why Multilevel Indexing?

Consider a file containing:

$$10,000 \text{ records}$$

Suppose each index entry occupies 4 bytes.

$$\text{Index Size} = 10000 \times 4$$

$$= \boxed{40000 \text{ bytes}}$$

Problem

The index itself may be too large for efficient searching or memory residency.

Solution

Create an index over the index.

Multilevel Indexing

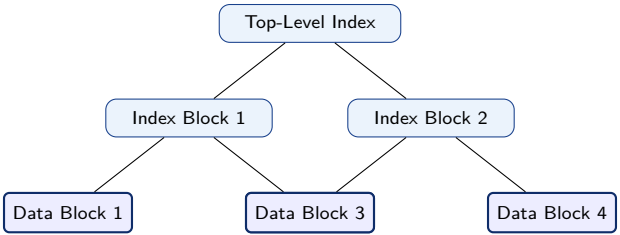
Definition

Multilevel indexing organizes index entries into multiple levels, where an upper-level index points to lower-level index blocks.



- Reduces the number of entries examined per level.
- Supports large data files.
- Introduces additional index levels.

Two-Level Index: Diagram



Observation

The top-level index points to lower-level index blocks, which identify data blocks.

Multilevel Indexing: Example

Given

- Data records = 10,000
- Index entry size = 8 bytes
- Index block size = 1024 bytes

Entries per index block:

$$\left\lfloor \frac{1024}{8} \right\rfloor = \boxed{128}$$

First-level blocks for a dense index:

$$\left\lceil \frac{10000}{128} \right\rceil = \boxed{79}$$

Second-level index:

$$\left\lceil \frac{79}{128} \right\rceil = \boxed{1 \text{ block}}$$

Multilevel Index: Search Process

1. Start at the top-level index.
2. Find the entry identifying the relevant lower-level index block.
3. Access the lower-level index.
4. Repeat until the data location is identified.
5. Retrieve the desired record.

Benefit

Searching a hierarchy avoids scanning a very large flat index sequentially.

Advantages of Multilevel Indexing

- Supports very large data files.
- Reduces the size of each searched index block.
- Provides hierarchical record lookup.
- Upper index levels may fit in memory.
- Forms the conceptual basis for tree-based indices.

Trade-off

Additional index levels require storage and maintenance.

Inverted Index

Definition

An inverted index maps a term or keyword to the documents or records in which it occurs.

Example

Keyword: `Cybersecurity`
Posting list:

$$\boxed{\{D_1, D_3, D_7, D_{10}\}}$$

The keyword appears in these four documents.

- Common in information retrieval.
- Supports keyword-based searching.

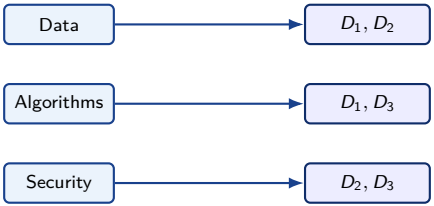
Inverted Index: Example

Suppose three documents contain:

- D_1: Data Structures and Algorithms
- D_2: Data Security
- D_3: Algorithms and Security

Keyword	Document List
Data	D_1, D_2
Structures	D_1
Algorithms	D_1, D_3
Security	D_2, D_3

Inverted Index: Diagram



Observation

Each keyword is associated with a list of documents containing that keyword.

Types of Inverted Indices

The notes describe two main variants.

Record-Level Inverted Index

Stores references to documents containing a word.

Word-Level Inverted Index

Stores document references and positions of word occurrences within documents.

Application

Word-level information supports phrase searches, such as locating “data structures” as a phrase.

Record-Level vs Word-Level Index

Feature	Record-Level	Word-Level
Stores	Document identifiers	Document identifiers and positions
Space	Generally lower	Generally higher
Keyword search	Supported	Supported
Phrase search	Needs additional processing	Can use position information
Complexity	Simpler	More detailed

Applications of Inverted Indices

- Search engines.
- Digital libraries.
- Document retrieval.
- Bibliographic databases.
- Keyword-based search systems.

Maintenance

When documents change, posting lists must also be updated.
The shared textbook emphasizes reorganization costs for traditional inverted-file structures.

Why B-Tree Indexing?

Large databases may contain millions of records.

- Entire databases may not fit in main memory.
- Flat indices can become very large.
- Disk access is costly.
- Balanced tree structures support efficient search and updates.

Key Idea

B-trees maintain a balanced search hierarchy designed to reduce disk accesses.

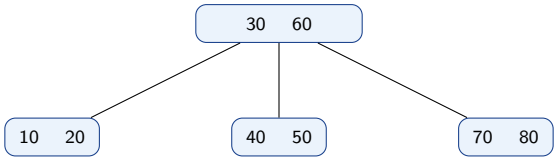
B-Tree Index

Definition

A B-tree is a balanced multiway search tree used to organize keys and associated record references.

- Nodes can contain multiple keys.
- Internal nodes direct searches.
- All leaf nodes occur at the same depth.
- Keys remain ordered.
- Insertions and deletions maintain balance.

Conceptual B-Tree Diagram



Search Example

Search for key 50:

1. Compare 50 with root keys 30 and 60.
2. Follow the middle child.
3. Locate 50 in the child node.

Advantages of B-Tree Indices

As discussed in the shared notes:

- Balanced tree height.
- Efficient exact-match queries.
- Support for range-based retrieval.
- Efficient insertion and deletion.
- Suitable for small and large tables.
- Reduced disk-access overhead.

Search Complexity

For n indexed keys:

$$\boxed{O(\log n)}$$

B-Tree vs Flat Index

Feature	Flat Ordered Index	B-Tree Index
Structure	Linear entries	Balanced hierarchy
Search	Depends on index access	Logarithmic height
Growth	May require restructuring	Tree grows through splits
Updates	May be costly	Balanced update operations
Disk use	Depends on layout	Designed for block-based access

What Comes Next?

Lecture 4: B-Trees

We will study:

1. B-tree properties.
2. Order and minimum degree.
3. Searching.
4. Insertion and node splitting.
5. Deletion and rebalancing.
6. Worked examples.
7. C implementation.
8. Time and space complexity.

Hashed Index

Definition

A hashed index uses a hash function to map search-key values to bucket addresses.

Let:

$$K = \text{set of search keys}$$

$$B = \text{set of bucket addresses}$$

A hash function maps:

$$\boxed{h : K \to B}$$

Example

$$h(k) = k \bmod 7$$

Hashed Index: Worked Example

Consider:

$$h(k) = k \bmod 5$$

Keys:

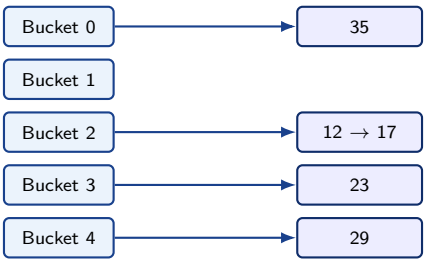
$$12,\ 17,\ 23,\ 29,\ 35$$

Key	Bucket
12	2
17	2
23	3
29	4
35	0

Observation

Keys 12 and 17 map to the same bucket, producing a collision.

Hash Buckets: Diagram



Collision Handling

Multiple records may be stored in a bucket, or overflow structures may be used.

Good Hash Function

The shared notes emphasize two characteristics.

1. Efficient average-case lookup.
2. Uniform distribution of records among available buckets.

Ideal Goal

Avoid concentrating many keys in the same bucket.

Worst Case

If all keys map to one bucket, search may degrade to:

$$\boxed{O(n)}$$

Hashed Index: Insertion

To insert a record with search key k:

1. Compute $h(k)$.
2. Locate the corresponding bucket.
3. Check available space.
4. Insert the record if space exists.
5. Otherwise, apply the collision-handling method.

Example

$$h(k) = k \bmod 5$$

For $k = 27$:

$$h(27) = 2$$

Insert the record into bucket 2.

Hashed Index: Searching

To search for key k:

1. Compute the bucket address $h(k)$.
2. Access the bucket.
3. Compare stored keys with k.
4. Search overflow records if necessary.
5. Return the matching record or report failure.

Expected Complexity

With suitable hashing and controlled collisions:

$$\boxed{O(1) \text{ average-case lookup}}$$

Hashed Index: Deletion

To delete a record with key k:

1. Compute $h(k)$.
2. Locate the corresponding bucket.
3. Search the bucket for the key.
4. Remove the matching record.
5. Update overflow links or bucket metadata if required.

Important

Deletion details depend on the collision resolution mechanism.

Limitations of Hashed Indices

- Collisions may reduce performance.
- Too many buckets can waste storage.
- Too few buckets increase collisions.
- Fixed bucket counts can become unsuitable as the file grows.
- Hash indices are generally less suitable for ordered range queries.

Design Consideration

The number of buckets and the hash function must be chosen carefully.

Ordered vs Hashed Index

Feature	Ordered Index	Hashed Index
Organization	Sorted keys	Hash buckets
Exact lookup	Efficient	Expected constant-time possible
Range queries	Well suited	Generally unsuitable
Collisions	Not applicable in hash sense	Possible
Maintenance	Ordered structure updates	Bucket and collision handling

Implementing a Simple Index in C

We will model an ordered index using:

- An array of search keys.
- An array of record positions.
- Binary search on sorted index entries.

Representation

$$(\text{Key}, \text{Record Position})$$

Scope

This example demonstrates an in-memory index. It is not a complete disk-based indexing system.

C Program: Index Structure

```c
#include <stdio.h>

typedef struct {
    int key;
    long recordPosition;
} IndexEntry;

int main(void) {
    IndexEntry index[] = {
        {101, 0},
        {102, 1},
        {103, 2},
        {104, 3},
        {105, 4}
    };

    int n = sizeof(index) / sizeof(index[0]);

    for (int i = 0; i < n; i++) {
        printf("Key: %d, Position: %ld\n",
               index[i].key,
               index[i].recordPosition);
    }

    return 0;
}
```

C Program: Binary Search on Index

```c
typedef struct {
    int key;
    long recordPosition;
} IndexEntry;

long searchIndex(IndexEntry a[], int n, int key) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid].key == key)
            return a[mid].recordPosition;

        if (a[mid].key < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}
```

Complexity

$$O(\log n)$$

C Program: Complete Index Search

```c
#include <stdio.h>

typedef struct {
    int key;
    long position;
} Entry;

long find(Entry a[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid].key == key)
            return a[mid].position;

        if (a[mid].key < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void) {
    Entry index[] = {
        {101, 0}, {102, 1}, {103, 2},
        {104, 3}, {105, 4}
    };

    long pos = find(index, 5, 104);

    if (pos >= 0)
        printf("Record position: %ld\n", pos);
    else
        puts("Record not found");
```

C Program: Dry Run

Search key:

$$K = 104$$

Index keys:

$$101,\ 102,\ 103,\ 104,\ 105$$

Step	Low	High	Mid Key
1	0	4	103
2	3	4	104

Record position $= 3$

Observation

Only two index comparisons are required.

C Program: Simple Hashed Index

```c
#include <stdio.h>

#define SIZE 7

int hash(int key) {
    return key % SIZE;
}

int main(void) {
    int keys[] = {10, 15, 22, 31, 40};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        printf("Key %d -> Bucket %d\n",
               keys[i], hash(keys[i]));
    }

    return 0;
}
```

Note

This demonstrates bucket calculation only. A full implementation also needs record storage and collision handling.

Classroom Exercise 1

Question

A file contains 20,000 records.
Each dense-index entry occupies 16 bytes.
Calculate the total dense-index size.

$$\text{Index Size} = n \times E$$

$$= 20000 \times 16$$

$$\boxed{320000 \text{ bytes}}$$

$$\boxed{312.5 \text{ KiB}}$$

Classroom Exercise 2

Question

A sorted file contains:

$$5, 10, 15, 20, 25, 30, 35, 40$$

The sparse index contains:

$$5, 20, 35$$

Explain how to locate key 30.

Solution

1. Select indexed key 20.
2. Follow its pointer.
3. Scan records 20, 25, and 30.
4. Stop when 30 is found.

Classroom Exercise 3

Question

A file contains 50,000 records.
Each index entry occupies 10 bytes.
Each index block has 1000 usable bytes.
How many first-level index blocks are required for a dense index?

Entries per block:

$$\frac{1000}{10} = 100$$

Number of blocks:

$$\frac{50000}{100} = \boxed{500}$$

Classroom Exercise 4

Question

For the hash function:

$$h(k) = k \bmod 7$$

Find the buckets for:

$$18,\ 25,\ 33,\ 42$$

Key	Bucket
18	4
25	4
33	5
42	0

Keys 18 and 25 collide.

MCQ 1

Which index stores an entry for every record in the textbook example?

1. Sparse index
2. Dense index
3. Cylinder index
4. Hash function

Correct Answer

Option 2: Dense index

MCQ 2

Which index is based on the field that determines the file's sequential order?

1. Secondary index
2. Inverted index
3. Primary index
4. Hashed index

Correct Answer

Option 3: Primary index

MCQ 3

Which index is commonly used for keyword-based document retrieval?

1. Inverted index
2. Relative index
3. Cylinder-only index
4. Sequential scan

Correct Answer

Option 1: Inverted index

MCQ 4

What is the main idea of multilevel indexing?

1. Delete the original index
2. Create an index over another index
3. Store all records in one block
4. Eliminate all pointers

Correct Answer

Option 2

MCQ 5

What happens when two keys map to the same hash bucket?

1. Binary search
2. Tree balancing
3. Collision
4. File deletion

Correct Answer

Option 3: Collision

Indexing: Overall Comparison

Index Type	Main Idea	Typical Use
Primary	Uses file-ordering key	Ordered file access
Secondary	Uses non-ordering key	Alternative searches
Dense	Entries for all records/keys	Direct index coverage
Sparse	Entries for selected locations	Smaller ordered index
Multilevel	Index over lower-level index	Large indices
Inverted	Term-to-document mapping	Text retrieval
B-Tree	Balanced multiway hierarchy	Database indexing
Hashed	Hash key to bucket	Exact-match lookup

Lecture Summary

- Indexing improves record retrieval.
- Ordered indices maintain sorted keys.
- Primary indices use the file-ordering field.
- Secondary indices support other search fields.
- Dense indices contain more entries than corresponding sparse indices.
- Sparse indices rely on ordered data.
- Multilevel indexing creates a hierarchy.
- Inverted indices support keyword retrieval.
- B-trees provide balanced indexing.
- Hashed indices use bucket addresses.

Questions for Revision

1. Define indexing and explain its advantages.
2. Differentiate primary and secondary indices.
3. Compare dense and sparse indices.
4. Explain sparse-index searching with an example.
5. Describe cylinder-surface indexing.
6. Explain multilevel indexing.
7. Differentiate record-level and word-level inverted indices.
8. Explain B-tree indexing advantages.
9. Describe insertion, search, and deletion in hashed indices.
10. Compare ordered and hashed indices.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 16, Section 16.7: Indexing.
2. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*.
3. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*.

Content Note

The indexing categories follow the shared notes. Additional numerical examples, diagrams, and C programs are instructional supplements.

Student Feedback

Share Your Feedback



Thank you!